Understand Object State vs Object Behavior (Interview Perspective)

> **Interview Goal:** This is a fundamental OOP concept. Interviewers ask it to determine whether you truly understand how Java models real-world entities. It is often followed by questions on **Encapsulation, Inheritance, Polymorphism, and Object Design**.

Learning Objectives

After this topic, you will be able to:

- Understand what object **state** is.
- Understand what object **behavior** is.
- Differentiate between state and behavior.
- Model real-world objects correctly.
- Answer common interview questions confidently.

---

1. What is an Object?

An **object** is an instance of a class that contains:

- **State (Data)**
- **Behavior (Actions)**

Example:

```
class Car {

    String color;
    int speed;

    void accelerate() {
        speed += 10;
    }

    void brake() {
        speed -= 10;
```

```
    }
}
```

Here:

State

```
color
speed
```

Behavior

```
accelerate()
brake()
```

---

2. Object State

State represents the **current data or properties** of an object.

In Java, **fields (instance variables)** represent the object's state.

Example

```
class Student {

    String name;
    int age;
    double cgpa;
}
```

State consists of:

```
name
age
cgpa
```

Each object can have a different state.

```
Student s1 = new Student();
s1.name = "Ali";
s1.age = 21;

Student s2 = new Student();
s2.name = "Sara";
s2.age = 20;
```

Same class

Different state.

---

3. Object Behavior

Behavior represents **what an object can do**.

In Java, behavior is defined by **methods**.

Example

```
class Student {

    String name;

    void study() {
        System.out.println(name + " is studying");
    }

    void attendClass() {
        System.out.println(name + " attended class");
    }
}
```

Behavior

```
study()
```

```
attendClass()
```

---

4. State and Behavior Work Together

Behavior usually **uses or changes** the object's state.

Example

```
class BankAccount {

    double balance;

    void deposit(double amount) {
        balance += amount;
    }

    void withdraw(double amount) {
        balance -= amount;
    }
}
```

Initially

```
balance = 1000
```

After

```
deposit(500);
```

State becomes

```
balance = 1500
```

Interview Insight

> **Methods don't just perform actions—they often modify the object's state.**

This is how real-world objects behave.

Real-World Examples

Object	State	Behavior
Car	color, speed, fuel	start(), accelerate(), brake()
Student	name, age, CGPA	study(), attendClass(), submitAssignment()
Bank Account	accountNumber, balance	deposit(), withdraw(), transfer()
Mobile Phone	battery, volume	call(), charge(), increaseVolume()

Interviewer's Reasoning

Interviewers don't ask this because they want definitions.

They're checking whether you know **how to model software**.

Suppose they ask:

> **Design a BankAccount class.**

Poor answer:

```
class BankAccount {

    double balance;

    void deposit(){}

    void withdraw(){}
}
```

Good answer:

Identify the **state** first.

```
Account Number
Balance
Account Holder
```

Then identify the **behavior**.

```
Deposit
Withdraw
Transfer
Check Balance
```

This shows you think like an object-oriented developer.

Common Misconceptions

Misconception 1

> State means local variables.

❌ Incorrect.

State is stored in **instance variables (fields)**.

Local variables exist only while a method executes.

Misconception 2

> Every method changes the object's state.

❌ Incorrect.

Example

```
void displayBalance() {
    System.out.println(balance);
}
```

This method only **reads** the state.

It doesn't modify it.

Misconception 3

> Every field should be modified directly.

❌ Bad practice.

Professional Java uses methods like

```
deposit()
withdraw()
setName()
```

instead of directly exposing fields.

This leads to **Encapsulation**, which you'll study next.

Interview Comparison

State vs Behavior

State	Behavior
Represented by fields	Represented by methods
Stores data	Performs actions
Changes over time	Reads or modifies state
Describes "what object has"	Describes "what object does"

Field vs Local Variable

Field	Local Variable
Represents object state	Temporary data
Lives as long as object exists	Exists only during method execution
Accessible by all instance methods	Accessible only inside its method

---

Tricky Interview Questions

Q1. Can two objects have the same behavior but different states?

Yes.

Example

```
Student s1 = new Student();
Student s2 = new Student();
```

Both can call

```
study();
```

But

```
Name
Age
CGPA
```

may differ.

Q2. Can two objects have the same state?

Yes.

```
Student s1 = new Student("Ali",21);
Student s2 = new Student("Ali",21);
```

They have identical state but are still **different objects**.

Q3. Does every method modify state?

No.

Some methods only read state.

Example

```
getBalance();
display();
toString();
```

Q4. Can an object exist without behavior?

Technically yes.

```
class Student {

    String name;
}
```

But from an OOP design perspective, it's generally not useful because objects are expected to encapsulate both **data and behavior**.

Q5. Can an object exist without state?

Yes.

Example

```
class Printer {

    void print() {
```

```
        System.out.println("Hello");
    }
}
```

Although possible, most real-world objects maintain some state.

Scenario-Based Interview Questions

Question 1

You are designing a **Car** class.

Identify the state and behavior.

Answer

State

```
brand
color
speed
fuelLevel
gear
```

Behavior

```
start()
stop()
accelerate()
brake()
changeGear()
```

Question 2

Why shouldn't balance in a BankAccount be public?

Expected Reasoning

If anyone can directly modify:

```
account.balance = -100000;
```

the object's state becomes invalid.

Instead,

```
withdraw()
deposit()
```

control how the state changes.

This is the motivation behind **Encapsulation**.

Question 3 (Very Common)

Why do methods belong inside the class instead of writing everything in main()?

Expected answer:

> Because methods represent the object's behavior. Keeping behavior inside the class groups the data and the operations on that data together, making the code easier to maintain, reuse, and protect. This is a core principle of Object-Oriented Programming.

Output Question

```
class Counter {

    int count = 0;

    void increment() {
        count++;
    }
}
```

```
public class Main {

    public static void main(String[] args) {

        Counter c = new Counter();

        c.increment();
        c.increment();

        System.out.println(c.count);
    }
}
```

Output

```
2
```

Interview Reasoning

- **State:** count
- **Behavior:** increment()
- Each call to increment() changes the object's state.

Frequently Asked Interview Questions

Easy

1. What is an object's state?
2. What is an object's behavior?
3. Which part of a class represents state?
4. Which part represents behavior?

Medium

5. Can behavior exist without state?
6. Can two objects have different states but the same behavior?
7. Does every method modify state?

Hard

8. Why should state usually be private?
9. Why should methods modify state instead of exposing fields directly?
10. How does understanding state vs behavior help implement Encapsulation?

⭐ Interview Insight (What Interviewers Are Actually Testing)

When an interviewer asks:

> **"What is the difference between object state and object behavior?"**

They're **not** looking for:

> "State is variables, behavior is methods."

They're evaluating whether you understand **object-oriented modeling**.

A strong answer is:

> "An object's **state** represents its current data, while its **behavior** represents the operations it can perform. Behavior often reads or modifies the state. When designing a class, I first identify the data the object needs to maintain, then define the operations that are allowed to interact with that data."

This demonstrates that you think beyond syntax and understand the core principles of Object-Oriented Programming—the kind of reasoning interviewers expect from a Java developer.